

User's Manual for the VSIIE solvers and Software Description

December 4, 2025

Abstract

This document provides a user's manual for the software that implements the different Variable Step-size Implicit Implicit Explicit (VSIIE) solvers for solving stiff systems of ordinary differential equations (ODEs) and also describes the implementation of the software.

1 User's manual

We have provided a user's manual to describe how to use the software and several python scripts which have been developed to perform automatically several experiments in order to compare the performance of the IIE (fixed stepsize) and the VSIIE (variable stepsize) methods.

1.1 Calling the solver

The executable of the solver can be generated by typing `make` in the main folder (usually named `VSIIE`) where the source code is located. This will generate the executable file called `IVP_Solver` in the same folder.

The solver can be called from the command line in a terminal window. The call follows the following format:

IVP_Solver *const_var problem_id. order Neqn tf stepsize tol num_reps alpha eta_min eta_max*

where:

- *const_var*: It denotes whether a constant (IIE) or a variable (VSIIE) stepsize method is used. The provided values can be:
 - `const`: constant stepsize (IIE method is used).
 - `var`: variable stepsize (VSIIE method is used).
- *problem_id.*: It denotes the mathematical model to be solved. The provided values can be:
 - `0`: 1D Advection-Diffusion.
 - `1`: Stiff Brusselator.

- 2: FKPP Combustion.
- 3: Ignition Combustion.
- 4: Fisher Combustion.
- *order*: It denotes the order of the VSIIE/IIE method to be used (1, 2, 3 or 4).
- *Neqn*: It denotes the number of ODEs in the system to be solved (It is important not to confuse this with the number of grid points.).
- *tf*: It denotes the final simulation time.
- *stepsize*: It denotes the constant stepsize if `const_var=const` (IIE method) and the initial stepsize if `const_var=var` (VSIIE method)
- *tol*: Error tolerance which it is only used for VSIIE (`const_var=var`)
- *num_reps*: Number of repetitions of the experiment (the experiment is repeated several times in order to obtain a report the minimum runtime of all repetitions).
- *alpha eta_min eta_max* (Optional): Parameters α , $\eta_{\min}$ and $\eta_{\max}$ of the adaptive time-stepping strategy (by default: $\alpha = 0.8$, $\eta_{\min} = 0.5$ and $\eta_{\max} = 4$).

Therefore, each call to the solver is applied to solve a particular IVP model instance using a particular method (VSIIE or IIE) of a given order.

When the solver is called, generates output by the screen and in output files. The first time the solver is called for a particular problem and final time, it creates a folder called `data` in the same folder where the executable is located and generates a binary file where the reference solution for this problem instance is saved. This reference solution is computed by using the VSIIE method of order 4 with a very small tolerance of 10^{-6} . Thus, the reference solution data can be reused in subsequent calls of the solver for the same problem instance, avoiding recomputation.

After running the solver, it generates a line with data in a text output file with `.txt` extension, whose name depends on the particular IVP model, method and final time. This output file is located in the same folder of the executable. The output file contains the following data in each line (separated by several spaces):

- For VSIIE: [tolerance] [error] [Minimum runtime] [Number of time steps]
- For IIE: [Constant stepsize] [Error] [Minimum runtime] [Number of time steps]

1.2 The scripts

There is a python script provided to run automatically the experiments corresponding to each particular mathematical IVP model and generate the corresponding plots (work-precision and time-steps). Each script runs the experiments for both IIE and VSIIE stepsize methods using different parameters. Each python script must be called in the particular folder where the script is located (although the user can call it from other locations if he creates a new folder, copies the script to the new folder and specifies what is the name of the new folder in the script). The particular scripts are:

- `adr.py`: It is located in the folder `ADR` and runs the experiments for the 1D Advection-Diffusion model.

- `sbr.py`: It is located in the folder `SBR` and runs the experiments for the Stiff Brusselator model.
- `fkpp.py`: It is located in the folder `FKPP` and runs the experiments for the FKPP Combustion model.
- `ignition.py`: It is located in the folder `IGNIT` and runs the experiments for the Ignition Combustion model.
- `fisher.py`: It is located in the folder `FISHER` and runs the experiments for the Fisher Combustion model.

For each python script, the user must define the following variables which define the experiments to be run:

- `Problem`: It denotes the problem id (0 to 4) corresponding to the mathematical IVP model.
- `IVP_name`: It denotes the name of the mathematical IVP model. The provided values can be:
 - `"1D_Adv-Diff"`: 1D Advection-Diffusion.
 - `"stiff_Brusselator"`: Stiff Brusselator.
 - `"FKPP"`: FKPP Combustion.
 - `"Ignition"`: Ignition Combustion.
 - `"Fisher"`: Fisher Combustion.
- `Neqn`: It denotes the number of ODEs in the system to be solved (It must be the the number of equations in the chosen IVP model, not the number of grid points).
- `Tf`: It denotes the final simulation time.
- `FOLDER`: It denotes the folder name where the experiments will be run (It must be the folder where the chosen python script is located). For instance, "ADR" for the 1D Advection-Diffusion model.
- `stepsize_array`: It denotes the 2D array of constant stepsizes to be used for the IIE methods of different orders. It is also used to define the initial stepsizes for the VSIIE methods of different orders. For instance, `stepsize_array[0]` contains the array of constant stepsizes for the first order IIE method and the initial stepsizes for the first order VSIIE method.
- `VSIIE_tol_array`: It denotes the array of error tolerances to be used for the VSIIE methods of different orders. For ADR, the definition of this array can be as follows:


```
stepsize_array = [
    [7.5e-3, 1e-3, 5e-4, 1.0e-4, 5e-5, 1e-5, 5e-6, 1e-6, 5e-7, 1e-7], #order1
    [5e-2, 2.5e-2, 1e-2, 7.5e-3, 5e-3, 1e-3, 5e-4, 1e-4, 5e-5, 1e-5, 5e-6, 1e-6], #order2
    [1e-1, 2.5e-2, 1e-2, 7.5e-3, 5e-3, 1e-3, 5e-4, 1e-4], #order3
    [1e-1, 7.5e-2, 5e-2, 2.5e-2, 1e-2, 7.5e-3, 5e-3, 1e-3, 5.0e-4]] #order4
```
- `VSIIE_tol_array`: It denotes the array of error tolerances to be used for the VSIIE methods of different orders. For ADR, the definition of this array can be as follows:

```

VSIIETol_array = [
    [1e-1, 5.0e-2, 1e-2, 1e-3, 1e-4, 1e-5, 1e-6, 1e-7, 1e-8, 1e-9], #order1
    [1e-1, 5.0e-2, 1e-2, 1e-3, 1e-4, 1e-5, 1e-6, 1e-7, 1e-8, 1e-9, 1e-10, 1e-11], #order2
    [2, 2.0e-01, 1.8e-03, 5.5e-04, 2e-04, 1.1E-06, 1e-07, 7e-10], #order3
    [5e-01, 2e-01, 1e-01, 4e-02, 1e-03, 3e-04, 4e-05, 4e-08, 1e-08]] #order4

```

- **alpha_array**: It denotes the array of values which indicates the α parameter values (which affects to the the time-stepping algorithm) to be used for each particular experiment of the VSIIIE methods of different orders. For ADR, the definition of this array can be as follows:

```

alpha_array = [
    [0.8, 0.8, 0.8, 0.8, 0.8, 0.8, 0.8, 0.8, 0.8, 0.8], #order1
    [0.4, 0.4, 0.4, 0.6, 0.6, 0.6, 0.6, 0.6, 0.6, 0.6, 0.6], #order2
    [0.3, 0.3, 0.4, 0.4, 0.4, 0.4, 0.4, 0.4], #order3
    [0.3, 0.3, 0.3, 0.3, 0.3, 0.3, 0.3, 0.3, 0.3], #order4

```

The arrays `stepsize_array`, `VSIIETol_array` and `alpha_array` must have the same number of elements for each order.

1.3 Calling the python scripts

Each python script can be called from the command line in a terminal window (being on the folder where the script is located). The call has the following format:

```
script_name.py VSIIETol_min_order VSIIETol_max_order IIE_min_order IIE_max_order N_Reps
```

where:

- *VSIIETol_min_order VSIIETol_max_order*: Minimum and maximum order of the VSIIIE methods to be used in the experiments.
- *IIE_min_order IIE_max_order* : Minimum and maximum order of the IIE methods to be used in the experiments.
- *N_Reps*: Number of repetitions of each experiment to obtain the minimum runtime.

If the user wish to run the experiments for all the orders of both VSIIIE and IIE methods with 3 repetitions, he can call the script as follows:

```
script_name.py 1 4 1 4 3.
```

On the other hand, if the user wish to run the experiments for a particular range of orders of VSIIIE and/or IIE methods, he can specify the corresponding minimum and maximum orders in the call to the script. So, for example, the following call

```
script_name.py 2 3 1 2 4
```

runs the experiments for orders 2 and 3 of the VSIIIE methods and orders 1 and 2 of the IIE methods using 4 repetitions.

One can also avoid easily to do experiments with a particular method (IIE or VSIIIE). So for example, the the call:

script_name.py 2 2 0 0 3

only runs the experiments for order 2 of the VSIIIE methods (there is no experiment for IIE methods). In this way, the scripts allows the user to choose what orders to be used in the experiments and thus avoid to recompute data which have been already obtained previously.

2 Adaptive time-stepping and error estimation

The algorithm of an adaptive time-stepping k -step VSIIIE method is given in Algorithm ?? . The new time stepsize is updated using the following formula:

$$\Delta t_{n+1} \leftarrow \min \left(\max \left(\alpha \left(\frac{tol}{\epsilon_i^c} \right)^{1/(p+1)}, \eta_{\min} \right), \eta_{\max} \right) \Delta t_{n+1}, \quad (1)$$

where p represents the order of the method and $\alpha \in (0, 1]$ represents a safety factor. In this work, we choose usually $\alpha = 0.8$, $range = tol/5$, $\eta_{\max} = 4.0$, and $\eta_{\min} = 0.5$. These values are used throughout this work unless specified otherwise.

Algorithm 1 Adaptive time-stepping method for a single time step of k -step VSIIIE method

```

 $\Delta t_{n+1} \leftarrow \text{next } \Delta t$  ▷ the initial  $\Delta t_{n+1}$  is next  $\Delta t$ , computed in the previous time step
 $accepted \leftarrow false$ 
while not  $accepted$  do ▷ Loop until the error is acceptable
     $y_{n+1}^c \leftarrow \text{TimeStep}(\Delta t_{n+1}, \Delta t_n, \dots, \Delta t_{n-k+2})$  ▷ Coarse step: TimeStep()
     $y_{n+1}^f \leftarrow \text{TimeStep}(\frac{\Delta t_{n+1}}{2}, \dots, \frac{\Delta t_{n+1} - \lfloor k/2 \rfloor}{2})$  ▷ Fine time step
     $\epsilon_i^c \leftarrow \text{Error}(\Delta t_{n+1}, \Delta t_n, \dots, \Delta t_{n-k+2}, y_{n+1}^c, y_{n+1}^f)$ 
     $accepted \leftarrow (\epsilon_i^c - tol) \leq range$  ▷ Check if  $\Delta t_{n+1}$  is accepted
     $factor \leftarrow \min \left( \max \left( \alpha \left( \frac{tol}{\epsilon_i^c} \right)^{1/(p+1)}, \eta_{\min} \right), \eta_{\max} \right)$ 
    if ( $accepted$ ) then ▷ If the error is acceptable
        if ( $|\epsilon_i^c - tol| \leq range$ ) then ▷ Update next  $\Delta t$  only if  $|\epsilon_i^c - tol| > range$ 
             $\text{next } \Delta t \leftarrow \Delta t_{n+1}$ 
        else
             $\text{next } \Delta t \leftarrow \Delta t_{n+1} \cdot factor$  ▷ Update next  $\Delta t$  for the next time step
        end if
    else ▷  $\Delta t_{n+1}$  is updated if error is not acceptable
         $\Delta t_{n+1} \leftarrow \Delta t_{n+1} \cdot factor$ 
    end if
end while
 $y_{n+1} \leftarrow y_{n+1}^c - \epsilon^c$ 

```

3 Description of the software

All the VSIIE methods and the IVP models have been implemented in a software framework using the C++ programming language. This software framework has been used to perform the experiments and follows a very similar structure to a previous software used to implement Variable Stepsize BDF methods (see [cite]). Additionally, several PYTHON scripts have been included to perform automatically the experiments associated to a particular IVP model. Here, we describe the main characteristics of the software framework and the scripts.

The software includes several subclasses to represent the IVP models, a class to implement the VSIIE solvers, a driver program to perform each particular experiment and the python scripts associated to that IVP model.

The driver program instantiates the different classes and calls the methods of the corresponding objects to run an experiment associated with a given IVP model generating the numerical error, the execution time and the number of stepsizes). In the driver program, the initialization of the intermediate pre-steps (from the initial conditions), which are necessary to start a particular VSIIE method, are performed by using the fourth order IMEX SBDF method with a constant stepsize of 10^{-6} or even less (although the initialization of the pre-steps which are required to use the IMEX SBDF-4 method is obtained by using the fourth-order explicit Runge-Kutta method with a constant small stepsize of 10^{-8}). The reference solution (which is used to compute the error) is also obtained in a similar way by using 4th order IMEX SBDF method using a stepsize of 10^{-6} .

The underlying code is publicly accessible as a GitHub repository with support that allows its easy and efficient installation on platforms running Linux.

3.1 Implementation of the IVP models

The IVPs representing the different mathematical models are implemented using a class hierarchy. In order to avoid redundancy, each IVP is defined as a subclass of a C++ abstract class called `IVP_ODE`. Class `IVP_ODE` defines the virtual functions to be implemented in the subclasses as well as functions which are common to all the IVPs (to compute $\mathbf{F}(t, \mathbf{y})$ from $\mathbf{f}(t, \mathbf{y})$ and $\mathbf{g}(t, \mathbf{y})$, to get the number of ODEs, to initialize the sparse matrices, etc.).

Each subclass defines the functions that are specific of each particular IVP model:

- A function to compute the initial conditions $\mathbf{y}(t_0)$.
- The functions required to compute the functions $\mathbf{f}(t, \mathbf{y})$ and $\mathbf{g}(t, \mathbf{y})$.
- The functions to compute the Jacobian matrix of $\mathbf{g}$ and $\mathbf{F}$.

In the current implementation, the sparse Jacobian matrices that are needed for each splitting approach are computed analytically by defining a specific function that depends on the particular IVP. However, we are working on defining an alternative generic function to approximate numerically the sparse Jacobian matrix without the need to explicitly program the function for each IVP model. This generic function would analyze the sparsity pattern of the Jacobian as a preliminary step in the computation.

3.2 Implementation of the VSIIE solvers

We have defined a specific C++ class to implement the VSIIE solvers with support for orders 1 to 4. The solvers apply adaptive time-stepping techniques to optimize the computational process where the accuracy

is ensured by error checking. The following public functions are highlighted in this class:

- `Time_Step` function describes the computations necessary to perform a time step of a k -th-order IIE method. This function has an argument which is a pointer to a `IVP_ODE` object.

This function uses functions of LIS (Library of Iterative Solvers) to perform certain operations with sparse matrices and iteratively solve linear systems. The function also calls functions of an implementation of the Basic Linear Algebra Subprograms (BLAS) library called *openblas* to perform various vector operations. A modified Newton iteration method is used to solve the nonlinear system that arises at each time step. To solve the sparse linear systems that arise, we have used the BiConjugate Gradient STABILized method (BICGSTAB) using the Incomplete LU factorization (ILUT) preconditioner because we have obtained good performance with these choices for the LIS iterative solver function.

- `Variable_Time_Step` function implements one coarse time step and two fine time steps followed by the approximation of the LTE (see Algorithm ??).
- `Adaptive_dt_SBDF_Integrate` implements the entire Algorithm ??.

3.3 The scripts